CardioAI — Relatório Técnico de Escalabilidade e Otimização

Desafio Ir Além 2: Processamento Assíncrono, Fila de Mensagens e Arquitetura Distribuída

FIAP — Fase 6: Coração Sob Controle | **Disciplina:** Otimização e Escalabilidade de Sistemas de IA (Cluster Computing)
Integrantes: Jonas Luis da Silva (RM561465) & Edson Henrique Felix Batista (RM566321)

1. Organização Modular Existente e Desacoplamento Técnico

O sistema CardioAI foi concebido de forma modular e desacoplada, separando nitidamente a inferência preditiva, o motor de regras, o quarteto de agentes e a persistência de sessões (conforme ilustrado em docs/diagrama_modulos.png):

• **Módulo Preditivo (src/predictor.py):** Carrega o artefato models/modelo_pico_risco.joblib (Random Forest de 200 árvores, n_samples=5.000). Executa normalização de features com tratamento de espaços, validação estrita das 10 variáveis e inferência de probabilidade.

• **Motor de Protocolos Clínicos (src/protocols.py):** Base institucional com 10 diretrizes hospitalares oficiais avaliadas de forma determinística (consultar_protocolos) para sinais vitais (SpO2 < 88%, FC > 120 bpm, PA < 90) e operacionais (carga > 85%, espera > 90 min).

• **Sistema Multiagente (src/agents_system/cardio_agents.py):** Agentes Orquestrador, Analista de Risco, Especialista em Protocolos e Planejador conectados por handoffs nativos assíncronos do OpenAI Agents SDK, protegidos por OutputGuardrailTripwireTriggered.

• **Orquestração e Persistência (src/agents_system/main.py):** Interface CLI com isolamento contextual (CardioContext) e sessões em SQLiteSession.

2. Estratégia de Simulação de Concorrência e Alta Carga (`fila_pacientes.py`)

Para simular picos hospitalares de triagem, o módulo fila_pacientes.py adota uma esteira assíncrona orientada a eventos:

• **Fila Assíncrona (asyncio.Queue):** Ingestão contínua de *N* pacientes sintéticos gerados por src.data_generation (com semente configurável).

• **Pool de *W* Workers Concorrentes:** Corrotinas paralelas consumindo a fila e executando o pipeline via Runner.run assíncrono.

• **Controle de Resiliência:** asyncio.Semaphore para respeitar cotas de chamadas simultâneas à API; timeout individual configurável por paciente; retry automático com backoff exponencial ($1.0 \times 2^{tentativa-1}$) em falhas transitórias de rede/rate limit; e isolamento de erros sem derrubar a fila.

• **Modo Lote Vetorizado (--lote):** Agrega todas as amostras em lote matricial e invoca predict_proba do Scikit-Learn de forma vetorizada.

3. Resultados Reais do Benchmark de Escalabilidade (`benchmark.py`)

A tabela a seguir consolida as medições reais obtidas no ambiente de testes (registradas em resultados/benchmark.csv e resultados/benchmark.png):

Cenário de Teste	Pacientes	Workers	Tempo Total	Média/Pac	Throughput	Erros
Fila (1w) — Sem LLM	1.000	1	14,09 s	14,1 ms	70,99 pac/s	0,0%
Fila (2w) — Sem LLM	1.000	2	10,92 s	21,8 ms	91,58 pac/s	0,0%
Fila (4w) — Sem LLM	1.000	4	13,40 s	53,5 ms	74,62 pac/s	0,0%
Fila (8w) — Sem LLM	1.000	8	12,76 s	101,8 ms	78,38 pac/s	0,0%
Lote Vetorizado (Batch ML)	**1.000**	**1**	**0,0128 s**	**< 0,1 ms**	**77.935,6 pac/s**	**0,0%**
Multiagente (1w) — LLM Real	10	1	85,13 s	8,51 s	0,12 pac/s	0,0%
Multiagente (2w) — LLM Real	10	2	40,28 s	7,87 s	0,25 pac/s	0,0%
Multiagente (4w) — LLM Real	**10**	**4**	**23,50 s**	**8,64 s**	**0,43 pac/s**	**0,0%**

4. Diagnóstico do Gargalo Computacional: ML Local vs LLM Remoto

Conclusão Crítica do Benchmark & Correção do Harness de Testes:
• **Sem LLM (Gargalo de CPU & GIL):** 2w aumentaram o throughput (71 ➔ 92 pac/s), mas 4w e 8w estagnaram (75 e 78 pac/s) devido à contenção do GIL no Random Forest. Solução ideal: **Lote Vetorizado** (77.935 pac/s) ou *multiprocessing*.
• **Com LLM (Escala Concorrente):** A expansão para 4 workers reduziu o tempo total de **85,13s para 23,50s** (aceleração de **3,62x**), comprovando a eficácia do asyncio em sobrepor tempos ociosos de I/O de rede.
• **Diagnóstico e Correção da Taxa de Erro:** Na primeira medição houve 10% de falha causada por um bug do próprio harness (event loop reutilizado e fechado entre chamadas de asyncio.run(), gerando Event loop is closed). O bug foi corrigido executando as rodadas em um único loop, e a nova medição teve **0,0% de erro** (100% de sucesso e 0 bloqueios de guardrails).

5. Arquitetura Distribuída para Ambientes de Alta Escala (Cluster Computing)

A partir dos gargalos diagnosticados, projeta-se a evolução do CardioAI para uma topologia corporativa distribuída (detalhada no diagrama arquitetural docs/diagrama_distribuido.png):

Camada da Arquitetura	Tecnologia Proposta	Papel Técnico e Impacto na Escalabilidade
API Gateway Stateless	FastAPI + NGINX Ingress (Container Docker)	Recebe eventos hospitalares (PEP/IoT), valida esquemas Pydantic e enfileira as mensagens de triagem sem reter conexões abertas, permitindo escala horizontal sem estado.
Fila de Mensagens Real	RabbitMQ / Apache Kafka / Redis Streams	Buffer elástico de absorção de picos de demanda com particionamento por prioridade clínica de atendimento e Dead Letter Queue (DLQ) para tratamento de anomalias sem perda de dados.
Workers em Cluster K8s	Kubernetes HPA + KEDA Autoscaler	Pool elástico de pods executando os agentes do OpenAI Agents SDK. O HPA provisiona instâncias dinamicamente com base no tamanho da fila e consumo de CPU, otimizando custo computacional.
Model Serving Desacoplado	Triton Inference Server / TorchServe / FastAPI ML	Isola o modelo Random Forest em microserviço dedicado com suporte a predição vetorizada em batching dinâmico via gRPC, liberando os workers de agentes de carregar pesos localmente.
Cache de Predições	Redis Cluster (In-Memory)	Armazena em cache (TTL de 60s) scores de sinais vitais estáveis de pacientes em monitoramento contínuo em leitos, evitando execuções repetidas do modelo e dos agentes.
Estado e Sessões Compartilhadas	PostgreSQL + Redis (substituindo SQLite)	Migração do SQLite local para banco relacional distribuído com réplicas de leitura e locks em Redis, permitindo que perguntas interativas sejam atendidas por qualquer worker do cluster.
Observabilidade Completa	Prometheus, Grafana, OpenTelemetry & Jaeger	Monitoramento em tempo real de taxas de throughput, latência P95/P99, tamanho da fila e distributed tracing completo para rastrear cada handoff do sistema multiagente.

Conclusão: A modularidade do CardioAI e a estratégia de desacoplamento entre inferência em lote (ML) e consumo assíncrono em fila (LLM) viabilizam a transição segura para ambientes corporativos distribuídos e supercomputadores hospitalares de alta performance.